Lecture 23
Machine Learning

Reinforcement Learning: Python Libraries

Off-the-Shelf Python Libraries for RL

Stable Baselines 3

stable-baselines-website

Gymnasium

gymnasium-website

Ray RLlib

ray-rllib-website

PyRLprob

pyrlprob-website

Recap: Markov Decision Process (MDP)

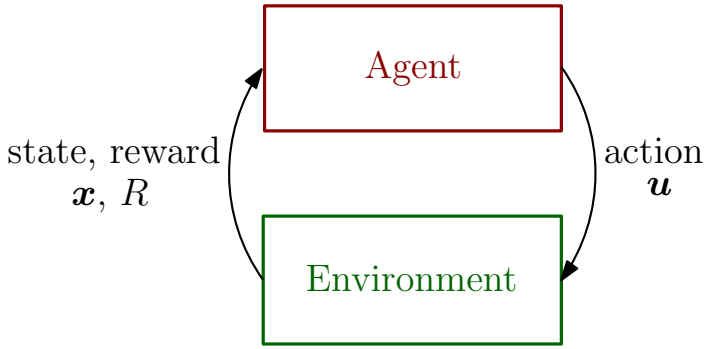
- The RL **environment**, i.e., the **control problem** the agent is solving, is formulated as a **Markov decision process** (MPD).

- An MDP is a **discrete-time** control process, where, at each time step, the agent chooses an action $\boldsymbol{u}$ based on the current state $\boldsymbol{x}$, causing the environment to transition to a new state $\boldsymbol{x}'$ and return a scalar reward R, which measures the "desirability" of the last agent's choice.

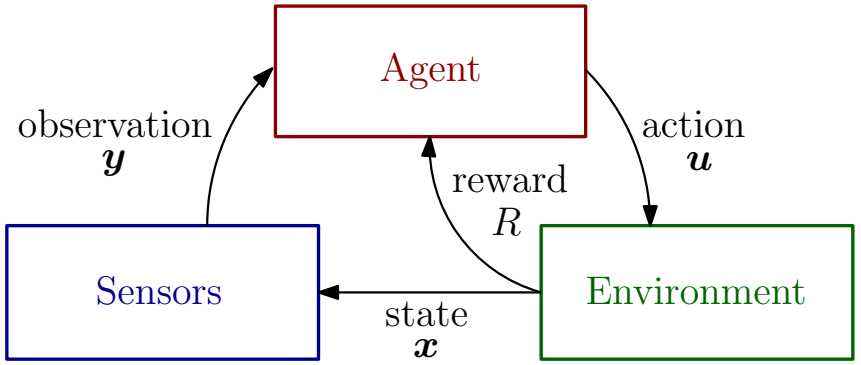
- An **MDP** is described by a **6-tuple**: $\langle X, U, \delta, \boldsymbol{x}_0, R, d \rangle$
  1. X is the **state space**
  2. U is **action space**
  3. $\delta: X \to U$ is the transition function, with $\boldsymbol{x}' = \delta(\boldsymbol{x}, \boldsymbol{u})$ the next state
  4. $\boldsymbol{x}_0$ is the initial **state**
  5. R: $X \times X \times U \to \mathbb{R}$ is the **reward** function, with $R = R(\boldsymbol{x}, \boldsymbol{u}, \boldsymbol{x}')$ the reward
  6. $d: X \to \{0,1\}$ is the **done signal**, indicated whether or not $\boldsymbol{x}$ is terminal

- If the MDP is just **partially observable** (POMDP), then we also need Y and z:
  7. Y the **observation space**
  8. $z: X \to Y$ the **observation or sensor model**, with $\boldsymbol{y} = z(\boldsymbol{x})$ the observation received in state $\boldsymbol{x}$

- The agent uses in general a **stochastic control policy** $\pi: U \times X \to [0,1]$, with $\pi(\boldsymbol{u}|\boldsymbol{x})$ expressing the probability of choosing action $\boldsymbol{u}$ in state $\boldsymbol{x}$.



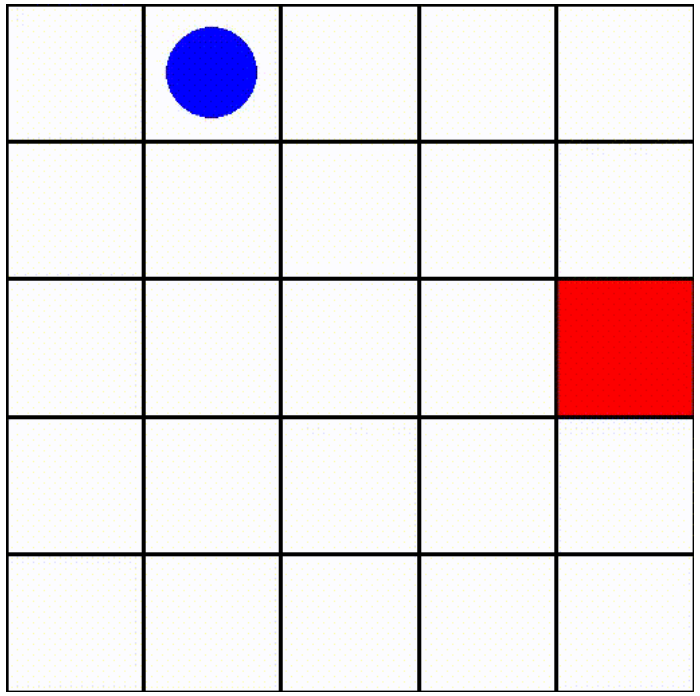
MDP



POMDP

Example Environments: Grid World

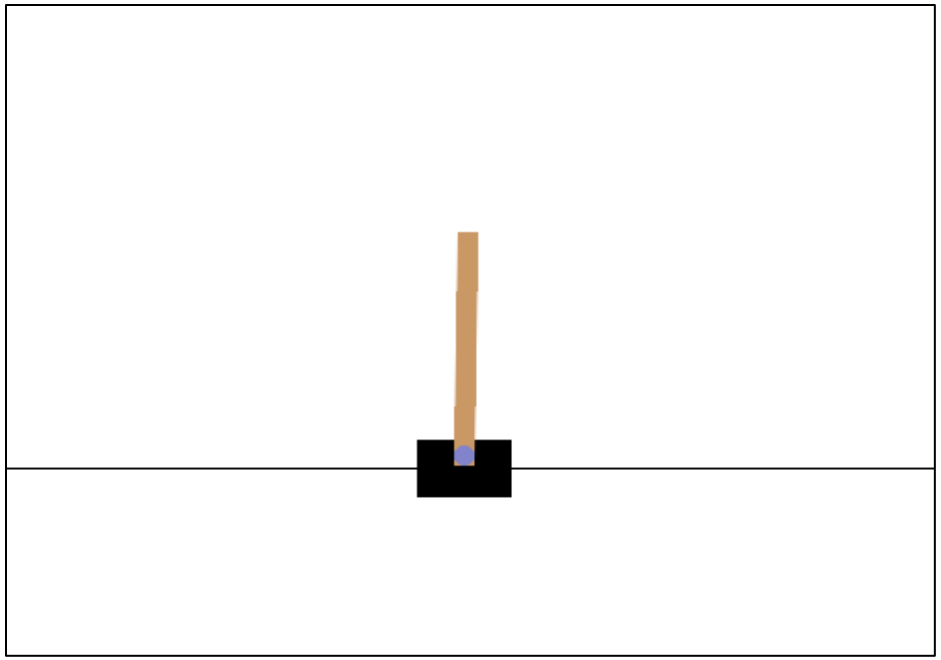
- A robot (the blue dot) has to find the shortest path to a target location (the red square), moving within a $s \times s$ square grid



- **State/Observation space**: 4 continuous variables:
  - *x,y* coordinates of the robot
  - *x,y* coordinates of the target square
- **Initial state/observation**: robot and target in random locations in the grid
- **Action space**: one discrete variable, with:
  - 0 = move right
  - 1 = move up
  - 2 = move left
  - 3 = move down
- **Transition function**: move to the corresponding square, unless there is a wall
- **Reward**:
  - -1 for each step/move
  - +10 if the target location is reached
- **Done signal**:
  - the target location is reached
  - number of steps greater than $2s^2$

Example Environments: Classic Control

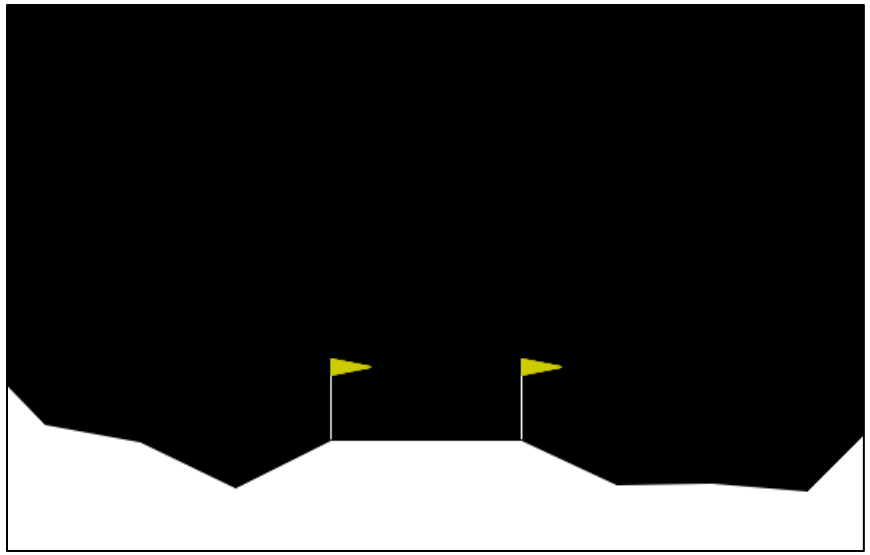
- A pole is attached by a joint to a cart, which moves along a track. The goal is to keep the pole upright by applying forces in the left and right direction on the cart, starting from a random position.



- **State/Observation space**: 3 continuous variables:
  - x coordinate of the cart
  - cart velocity $\dot{x}$
  - pole angle θ
  - pole angular velocity $\dot{\theta}$
- **Initial state/observation**: all variables have a random value in $(-0.05, 0.05)$
- **Action space**: one binary variable, with:
  - 0 = push cart to the left
  - 1 = push cart to the right
- **Transition function**: equations of motion are integrated, given the chosen control
- **Reward**:
  - +1 for every step taken
- **Done signal**:
  - pole angle greater than $\pm 12\ deg$
  - cart position greater than $\pm 2.4\ m$
  - number of steps greater than 500

Example Environments: Box2D

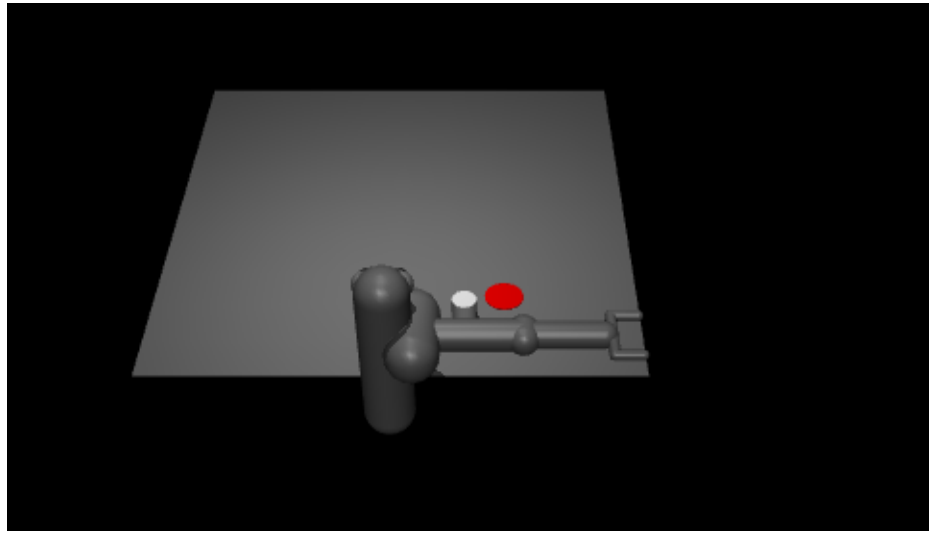
- A 2-D lander with unlimited fuel has to realize a soft landing within the landing pad delimited by the yellow flags, by thrusting with one of three engines.

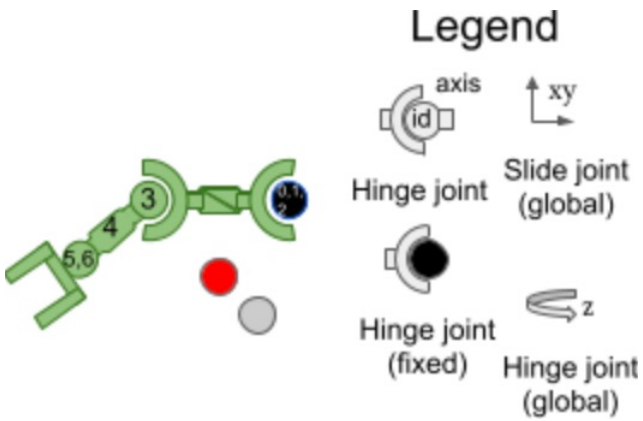


- **State/Observation space**: 8 continuous/Boolean variables:
  - x, y coordinates of the lander
  - $\dot{x}, \dot{y}$ linear velocities
  - Angle θ
  - angular velocity $\dot{\theta}$
  - two booleans that represent whether each leg is in contact with the ground or not
- **Initial state/observation**: the lander starts at the top center of the viewport with a random initial force applied to its center of mass
- **Action space**: one discrete variable, with:
  - 0 = do nothing
  - 1= fire left engine
  - 2 = fire main engine
  - 3 = fire right engine
- **Transition function**: equations of motion are integrated, given the chosen control
- **Reward**: for each step, the reward:
  - is increased/decreased the closer/further the lander is to the landing pad
  - is increased/decreased the slower/faster the lander is moving.
  - is decreased the more the lander is tilted (angle not horizontal).
  - is increased by 10 points for each leg that is in contact with the ground.
  - is decreased by 0.03/0.3 points each step a side engine/the main engine is firing
  - The episode receive an additional reward of -100/+100 points for crashing/landing safely.
- **Done signal**:
  - the lander crashes
  - the lander goes outside of the viewport ($x > 1$)

Example Environments: MuJoCo (Multi-Joint dynamics with Contact)

- "Pusher" is a multi-jointed robot arm that is very similar to a human arm. The goal is to move a target cylinder (called object) to a goal position using the robot's end effector (called fingertip). The robot consists of shoulder, elbow, forearm and wrist joints.

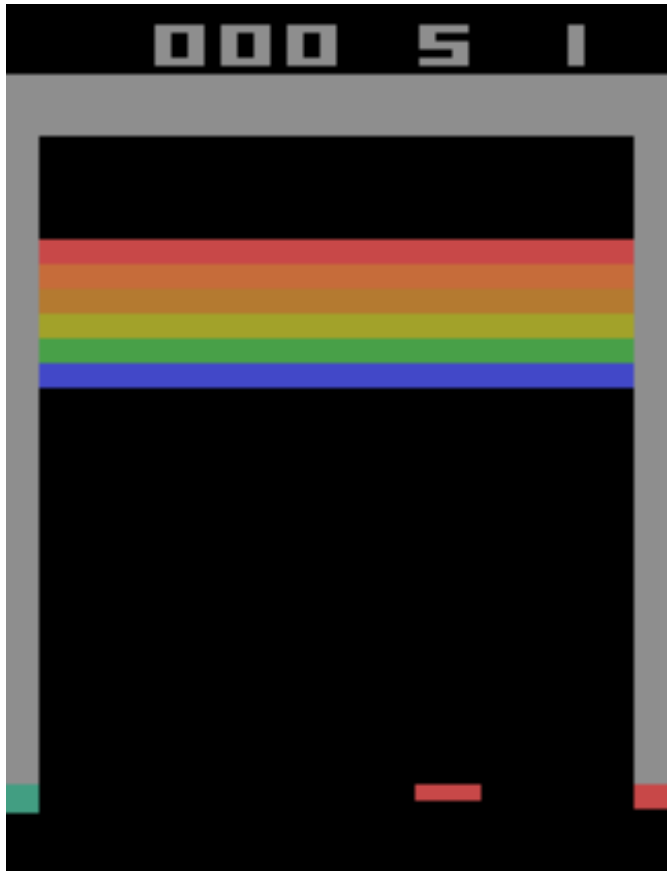




- **State/Observation space**: 23 continuous variables:
  - 7 position values of the robot's body parts
  - 7 velocities of these individual body parts (their derivatives)
  - 3 coordinates of the fingertip of the pusher
  - 3 coordinates of the object to be moved
  - 3 coordinates of the goal position
- **Initial state/observation**: object in a random location, arm has a random velocity
- **Action space**: 7 continuous variable, with:
  - Rotation of the panning the shoulder (torque)
  - Rotation of the shoulder lifting joint (torque)
  - Rotation of the shoulder rolling joint (torque)
  - Rotation of hinge joint that flexed the elbow (torque)
  - Rotation of hinge that rolls the forearm (torque)
  - Rotation of flexing the wrist (torque)
  - Rotation of rolling the wrist (torque)
- **Transition function**: equations of motion are integrated, given the chosen control
- **Reward**: $R = -0.5\, R_{near} - R_{dist} - 0.1 R_{ctrl}$, where:
  - R_{near} = distance of the fingertip from the object
  - R_{dist} = distance of the object from the target position
  - R_{ctrl} = norm of the control
- **Done signal**:
  - number of steps greater than 100

Example Environments: Atari Breakout

- A classic Atari game! You move a paddle and hit the ball, shooting it toward a brick wall at the top of the screen. Your goal is to destroy the brick wall, with five total lives (one life = one ball miss)



- **Observation space**:
  - 4 stacked rgb images of 260x160x3 pixels each, collected at 4 consecutive steps
- **Initial observation**: paddle and ball in random locations, no bricks destroyed
- **Action space**: one discrete variable, with:
  - 0 = do nothing
  - 1 = fire
  - 2 = move right
  - 3 = move left
- **Transition function**: the image is updated based on the new position of the paddle and ball
- **Reward**:
  - +1 when a blue or green brick is destroyed
  - + 4 when a yellow or gold brick is destroyed
  - +7 when an orange or red brick is destroyed
- **Done signal**:
  - The paddle misses the ball five times